Properties of GPGPU

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

int main()
{

    int deviceCount;
    cudaGetDeviceCount(&deviceCount);
    if (deviceCount == 0) {
        printf("There is no device supporting CUDA\n");
        return 0;
    }

    for (int dev = 0; dev < deviceCount; ++dev) {
        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, dev);

        if (dev == 0) {
            if (deviceProp.major < 1)
                printf("There is no device supporting CUDA.\n");
            else if (deviceCount == 1)
                printf("There is 1 device supporting CUDA\n");
            else
                printf("There are %d devices supporting CUDA\n",
deviceCount);
        }

        printf("\nDevice %d: \"%s\"\n", dev, deviceProp.name);
        printf("  Major revision number:                         %d\n",
deviceProp.major);
        printf("  Minor revision number:                         %d\n",
deviceProp.minor);
        printf("  Total amount of global memory:                 %zu
bytes\n", deviceProp.totalGlobalMem);
        printf("  Total amount of constant memory:               %zu
bytes\n", deviceProp.totalConstMem);
        printf("  Total amount of shared memory per block:       %zu
bytes\n", deviceProp.sharedMemPerBlock);
        printf("  Total number of registers per block:           %d\n",
deviceProp.regsPerBlock);
        printf("  Warp size:                                     %d\n",
deviceProp.warpSize);
        printf("  Multiprocessor count:                          %d\n",
deviceProp.multiProcessorCount);
```

```
        printf("  Maximum threads per block:                     %d\n",
deviceProp.maxThreadsPerBlock);
        printf("  Maximum block dimensions:                      %d x %d x
%d\n",
               deviceProp.maxThreadsDim[0], deviceProp.maxThreadsDim[1],
deviceProp.maxThreadsDim[2]);
        printf("  Maximum grid dimensions:                       %d x %d x
%d\n",
               deviceProp.maxGridSize[0], deviceProp.maxGridSize[1],
deviceProp.maxGridSize[2]);
        printf("  Memory pitch:                                  %zu
bytes\n", deviceProp.memPitch);
        printf("  Texture alignment:                             %zu
bytes\n", deviceProp.textureAlignment);
        printf("  Clock rate:                                    %d
kilohertz\n", deviceProp.clockRate);
    }
}
```

```
There is 1 device supporting CUDA

Device 0: "Tesla T4"
  Major revision number:                         7
  Minor revision number:                         5
  Total amount of global memory:                 15828320256 bytes
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers per block:           65536
  Warp size:                                     32
  Multiprocessor count:                          40
  Maximum threads per block:                     1024
  Maximum block dimensions:                      1024 x 1024 x 64
  Maximum grid dimensions:                       2147483647 x 65535 x 65535
  Memory pitch:                                  2147483647 bytes
  Texture alignment:                             512 bytes
  Clock rate:                                    1590000 kilohertz
```

Each Thread Prints its ID along with Hello World. Launch Kernel with One Block and Multiple Threads.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void hello_kernel() {
    // thread id inside the grid (for one block this equals threadIdx.x)
```

```
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    printf("Thread %d: hello world\n", tid);
}

int main(void) {
    const int threadsPerBlock = 8; // change to any reasonable number (e.g.,
32, 128, 256)
    const int blocks = 1;

    // Launch kernel
    hello_kernel<<<blocks, threadsPerBlock>>>();

    // Check for launch errors
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        fprintf(stderr, "Kernel launch failed: %s\n",
cudaGetErrorString(err));
        return 1;
    }

    // Wait for device to finish and flush device printf buffer
    err = cudaDeviceSynchronize();
    if (err != cudaSuccess) {
        fprintf(stderr, "cudaDeviceSynchronize returned error: %s\n",
cudaGetErrorString(err));
        return 1;
    }

    return 0;
}
```

Each Thread Prints its ID along with Hello World. Launch Kernel with multiple blocks and multiple threads.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void hello_kernel() {
    // Thread index within block
    int local_tid = threadIdx.x;
    // Block index within grid
    int block_id = blockIdx.x;
    // Threads per block
    int threads_per_block = blockDim.x;
    // Compute global thread ID across all blocks
    int global_tid = block_id * threads_per_block + local_tid;
```

```
    printf("Block %d, Thread %d (Global ID %d): Hello, World!\n",
           block_id, local_tid, global_tid);
}

int main(void) {
    // Set grid and block dimensions
    int threadsPerBlock = 4; // threads per block
    int blocks = 3;          // number of blocks

    // Launch the kernel
    hello_kernel<<<blocks, threadsPerBlock>>>();

    // Check for errors
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        fprintf(stderr, "Kernel launch failed: %s\n",
cudaGetErrorString(err));
        return 1;
    }

    // Synchronize to ensure output is printed
    err = cudaDeviceSynchronize();
    if (err != cudaSuccess) {
        fprintf(stderr, "Device sync failed: %s\n",
cudaGetErrorString(err));
        return 1;
    }

    return 0;
}
```

Each Thread Prints its ID along with Hello World. Launch Kernel with 2D blocks and 2D threads.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void hello_kernel_2d() {
    // Coordinates of the thread within its block
    int local_x = threadIdx.x;
    int local_y = threadIdx.y;

    // Coordinates of the block within the grid
    int block_x = blockIdx.x;
    int block_y = blockIdx.y;
```

```
    // Dimensions of a block
    int block_dim_x = blockDim.x;
    int block_dim_y = blockDim.y;

    // Compute global coordinates across the whole grid
    int global_x = block_x * block_dim_x + local_x;
    int global_y = block_y * block_dim_y + local_y;

    // Flatten 2D coordinates into a single global thread ID
    int global_id = global_y * (gridDim.x * block_dim_x) + global_x;

    printf("Block(%d,%d), Thread(%d,%d) -> Global(%d,%d) [ID %d]: Hello,
World!\n",
           block_x, block_y, local_x, local_y, global_x, global_y,
global_id);
}

int main(void) {
    // Define a 2D block of threads (e.g., 4×4 = 16 threads per block)
    dim3 threadsPerBlock(4, 4);

    // Define a 2D grid of blocks (e.g., 3×2 = 6 blocks)
    dim3 numBlocks(3, 2);

    // Launch the kernel
    hello_kernel_2d<<<numBlocks, threadsPerBlock>>>();

    // Synchronize to make sure printf output appears
    cudaDeviceSynchronize();

    return 0;
}
```

```
dim3 threadsPerBlock(4, 4);  // blockDim.x = 4, blockDim.y = 4
dim3 numBlocks(3, 2);        // gridDim.x = 3, gridDim.y = 2
```

Total threads overall = `(3 * 4) × (2 * 4) = 12 × 8 = 96 threads.`

```
Block coordinates: (blockIdx.x, blockIdx.y) = (1, 1)
Thread coordinates: (threadIdx.x, threadIdx.y) = (2, 1)
```

```
global_x = blockIdx.x * blockDim.x + threadIdx.x = 1 * 4 + 2 = 6
global_y = blockIdx.y * blockDim.y + threadIdx.y = 1 * 4 + 1 = 5
```

```
global_id = global_y * (gridDim.x * blockDim.x) + global_x = (5 * (3 * 4)) +
6 = 66
```

Vector Addition (1D)

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>
#include <time.h>

// CUDA kernel for vector addition
__global__ void vectorAddKernel(const float *A, const float *B, float *C,
int N) {
    int globalIndex = blockIdx.x * blockDim.x + threadIdx.x; // global
thread ID
    if (globalIndex < N)
        C[globalIndex] = A[globalIndex] + B[globalIndex];
}

// Initialize host vectors with random values
void initializeVectors(float *A, float *B, int N) {
    for (int i = 0; i < N; i++) {
        A[i] = (float)(rand() % 100) / 10.0f;
        B[i] = (float)(rand() % 100) / 10.0f;
    }
}

// CPU implementation of vector addition
void vectorAddCPU(const float *A, const float *B, float *C, int N) {
    for (int i = 0; i < N; i++)
        C[i] = A[i] + B[i];
}

int main() {
    int N = 1e6;  // 1 million elements
    size_t bytes = N * sizeof(float);

    // Host (CPU) memory allocation
    float *h_A = (float *)malloc(bytes);
    float *h_B = (float *)malloc(bytes);
    float *h_C_CPU = (float *)malloc(bytes);
    float *h_C_GPU = (float *)malloc(bytes);

    srand(time(NULL));
    initializeVectors(h_A, h_B, N);
```

```
    // --- CPU computation ---
    clock_t startCPU = clock();
    vectorAddCPU(h_A, h_B, h_C_CPU, N);
    clock_t endCPU = clock();
    double cpuTime = (double)(endCPU - startCPU) / CLOCKS_PER_SEC;

    // --- GPU computation setup ---
    float *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, bytes);
    cudaMalloc((void **)&d_B, bytes);
    cudaMalloc((void **)&d_C, bytes);

    cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;

    cudaEvent_t startGPU, stopGPU;
    cudaEventCreate(&startGPU);
    cudaEventCreate(&stopGPU);
    cudaEventRecord(startGPU);

    // --- Launch kernel ---
    vectorAddKernel<<<numBlocks, threadsPerBlock>>>(d_A, d_B, d_C, N);

    cudaEventRecord(stopGPU);
    cudaEventSynchronize(stopGPU);

    cudaMemcpy(h_C_GPU, d_C, bytes, cudaMemcpyDeviceToHost);

    float gpuTime = 0.0f;
    cudaEventElapsedTime(&gpuTime, startGPU, stopGPU);
    gpuTime /= 1000.0f;  // convert ms to seconds

    // --- Results ---
    printf("\n=== Vector Addition using CUDA ===\n");
    printf("Number of Elements: %d\n", N);
    printf("CPU Execution Time: %.6f s\n", cpuTime);
    printf("GPU Execution Time: %.6f s\n", gpuTime);
    printf("Speedup (CPU/GPU): %.2fx\n", cpuTime / gpuTime);

    // --- Cleanup ---
    free(h_A);
    free(h_B);
    free(h_C_CPU);
    free(h_C_GPU);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
```

```
    return 0;
}
```

Matrix Addition

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>
#include <time.h>

__global__ void matrixAdd(float *A, float *B, float *C, int M, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int idx = row * N + col;
    if (row < M && col < N)
        C[idx] = A[idx] + B[idx];
}

void initializeMatrix(float *A, float *B, int M, int N) {
    for (int i = 0; i < M * N; i++) {
        A[i] = (float)(rand() % 100) / 10.0;
        B[i] = (float)(rand() % 100) / 10.0;
    }
}

void matrixAddCPU(float *A, float *B, float *C, int M, int N) {
    for (int i = 0; i < M * N; i++)
        C[i] = A[i] + B[i];
}

int main() {
    int M = 1000, N = 1000;
    size_t size = M * N * sizeof(float);

    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float *h_C_cpu = (float *)malloc(size);
    float *h_C_gpu = (float *)malloc(size);

    srand(time(NULL));
    initializeMatrix(h_A, h_B, M, N);

    // --- CPU computation ---
    clock_t start_cpu = clock();
    matrixAddCPU(h_A, h_B, h_C_cpu, M, N);
```

```
    clock_t end_cpu = clock();
    double cpu_time = ((double)(end_cpu - start_cpu)) / CLOCKS_PER_SEC;

    // --- GPU computation setup ---
    float *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    dim3 blockSize(16, 16);
    dim3 numBlocks((N + blockSize.x - 1) / blockSize.x,
                   (M + blockSize.y - 1) / blockSize.y);

    cudaEvent_t start_gpu, stop_gpu;
    cudaEventCreate(&start_gpu);
    cudaEventCreate(&stop_gpu);
    cudaEventRecord(start_gpu);

    // --- Kernel launch ---
    matrixAdd<<<numBlocks, blockSize>>>(d_A, d_B, d_C, M, N);

    cudaEventRecord(stop_gpu);
    cudaEventSynchronize(stop_gpu);

    cudaMemcpy(h_C_gpu, d_C, size, cudaMemcpyDeviceToHost);

    float gpu_time = 0;
    cudaEventElapsedTime(&gpu_time, start_gpu, stop_gpu);
    gpu_time /= 1000.0;

    // --- Print performance results ---
    printf("\n=== Matrix Addition (GPU vs CPU) ===\n");
    printf("Matrix Size: %dx%d\n", M, N);
    printf("CPU Time: %.6f s\n", cpu_time);
    printf("GPU Time: %.6f s\n", gpu_time);
    printf("Speedup (CPU/GPU): %.2fx\n", cpu_time / gpu_time);

    // --- Cleanup ---
    free(h_A);
    free(h_B);
    free(h_C_cpu);
    free(h_C_gpu);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
```

```
    return 0;
}
```

Dot Product

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>
#include <time.h>

// CUDA kernel to compute partial dot products
__global__ void dotProductKernel(float *A, float *B, float *result, int N) {
    __shared__ float cache[256];  // shared memory for reduction
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int cacheIndex = threadIdx.x;

    float temp = 0.0f;

    // Each thread computes part of the dot product
    while (tid < N) {
        temp += A[tid] * B[tid];
        tid += blockDim.x * gridDim.x; // move to next element handled by
same thread
    }

    cache[cacheIndex] = temp;  // store in shared memory
    __syncthreads();

    // Reduction within the block
    int i = blockDim.x / 2;
    while (i != 0) {
        if (cacheIndex < i)
            cache[cacheIndex] += cache[cacheIndex + i];
        __syncthreads();
        i /= 2;
    }

    // Add the block's result to the global result
    if (cacheIndex == 0)
        atomicAdd(result, cache[0]);
}

// Initialize vectors with random float values
void initializeVectors(float *A, float *B, int N) {
    for (int i = 0; i < N; i++) {
        A[i] = (float)(rand() % 100) / 10.0f;
```

```
        B[i] = (float)(rand() % 100) / 10.0f;
    }
}

// CPU reference computation
float dotProductCPU(float *A, float *B, int N) {
    float sum = 0.0f;
    for (int i = 0; i < N; i++)
        sum += A[i] * B[i];
    return sum;
}

int main() {
    int N = 100000;  // vector size
    size_t size = N * sizeof(float);

    float *h_A = (float *)malloc(size);
    float *h_B = (float *)malloc(size);
    float h_result_cpu = 0.0f, h_result_gpu = 0.0f;

    srand(time(NULL));
    initializeVectors(h_A, h_B, N);

    // --- CPU computation ---
    clock_t start_cpu = clock();
    h_result_cpu = dotProductCPU(h_A, h_B, N);
    clock_t end_cpu = clock();
    double cpu_time = ((double)(end_cpu - start_cpu)) / CLOCKS_PER_SEC;

    // --- GPU computation ---
    float *d_A, *d_B, *d_result;
    cudaMalloc((void **)&d_A, size);
    cudaMalloc((void **)&d_B, size);
    cudaMalloc((void **)&d_result, sizeof(float));

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_result, &h_result_gpu, sizeof(float),
cudaMemcpyHostToDevice);

    int blockSize = 256;
    int numBlocks = 256;

    cudaEvent_t start_gpu, stop_gpu;
    cudaEventCreate(&start_gpu);
    cudaEventCreate(&stop_gpu);
    cudaEventRecord(start_gpu);

    dotProductKernel<<<numBlocks, blockSize>>>(d_A, d_B, d_result, N);
    cudaDeviceSynchronize();
```

```
    cudaEventRecord(stop_gpu);
    cudaEventSynchronize(stop_gpu);

    cudaMemcpy(&h_result_gpu, d_result, sizeof(float),
cudaMemcpyDeviceToHost);

    float gpu_time = 0.0f;
    cudaEventElapsedTime(&gpu_time, start_gpu, stop_gpu);
    gpu_time /= 1000.0f; // convert ms to seconds

    // --- Results ---
    printf("\n=== Dot Product Computation ===\n");
    printf("Vector Size: %d\n", N);
    printf("CPU Result: %.4f\n", h_result_cpu);
    printf("GPU Result: %.4f\n", h_result_gpu);
    printf("CPU Time: %.6f s\n", cpu_time);
    printf("GPU Time: %.6f s\n", gpu_time);
    printf("Speedup (CPU/GPU): %.2fx\n", cpu_time / gpu_time);

    // --- Cleanup ---
    free(h_A);
    free(h_B);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_result);

    return 0;
}
```

Matrix Multiplication

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>
#include <time.h>

// GPU Kernel: Matrix Multiplication
__global__ void matrixMultiply(float *A, float *B, float *C, int M, int N,
int P) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;  // Row index of C
    int col = blockIdx.x * blockDim.x + threadIdx.x;  // Column index of C

    if (row < M && col < P) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k)
```

```
            sum += A[row * N + k] * B[k * P + col];
        C[row * P + col] = sum;
    }
}

// Initialize a matrix with random float values
void initializeMatrix(float *A, int rows, int cols) {
    for (int i = 0; i < rows * cols; ++i)
        A[i] = (float)(rand() % 100) / 10.0f;
}

// CPU function for matrix multiplication
void matrixMultiplyCPU(float *A, float *B, float *C, int M, int N, int P) {
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < P; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < N; ++k)
                sum += A[i * N + k] * B[k * P + j];
            C[i * P + j] = sum;
        }
    }
}

int main() {
    // Define matrix dimensions: A(MxN), B(NxP), C(MxP)
    int M = 500, N = 500, P = 500;
    size_t size_A = M * N * sizeof(float);
    size_t size_B = N * P * sizeof(float);
    size_t size_C = M * P * sizeof(float);

    // Allocate host memory
    float *h_A = (float *)malloc(size_A);
    float *h_B = (float *)malloc(size_B);
    float *h_C_cpu = (float *)malloc(size_C);
    float *h_C_gpu = (float *)malloc(size_C);

    // Initialize input matrices
    srand(time(NULL));
    initializeMatrix(h_A, M, N);
    initializeMatrix(h_B, N, P);

    // --- CPU Computation ---
    clock_t start_cpu = clock();
    matrixMultiplyCPU(h_A, h_B, h_C_cpu, M, N, P);
    clock_t end_cpu = clock();
    double cpu_time = ((double)(end_cpu - start_cpu)) / CLOCKS_PER_SEC;

    // --- GPU Computation ---
    float *d_A, *d_B, *d_C;
    cudaMalloc((void **)&d_A, size_A);
```

```
    cudaMalloc((void **)&d_B, size_B);
    cudaMalloc((void **)&d_C, size_C);

    cudaMemcpy(d_A, h_A, size_A, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size_B, cudaMemcpyHostToDevice);

    // Configure grid and block dimensions
    dim3 blockSize(16, 16);
    dim3 numBlocks((P + blockSize.x - 1) / blockSize.x,
                   (M + blockSize.y - 1) / blockSize.y);

    cudaEvent_t start_gpu, stop_gpu;
    cudaEventCreate(&start_gpu);
    cudaEventCreate(&stop_gpu);
    cudaEventRecord(start_gpu);

    // Launch kernel
    matrixMultiply<<<numBlocks, blockSize>>>(d_A, d_B, d_C, M, N, P);
    cudaDeviceSynchronize();

    cudaEventRecord(stop_gpu);
    cudaEventSynchronize(stop_gpu);

    // Copy result back to host
    cudaMemcpy(h_C_gpu, d_C, size_C, cudaMemcpyDeviceToHost);

    // Measure GPU time
    float gpu_time = 0.0f;
    cudaEventElapsedTime(&gpu_time, start_gpu, stop_gpu);
    gpu_time /= 1000.0f; // Convert ms to seconds

    // --- Results ---
    printf("\n=== Matrix Multiplication (CPU vs GPU) ===\n");
    printf("Matrix A: %dx%d\n", M, N);
    printf("Matrix B: %dx%d\n", N, P);
    printf("Matrix C: %dx%d\n", M, P);
    printf("CPU Time: %.6f s\n", cpu_time);
    printf("GPU Time: %.6f s\n", gpu_time);
    printf("Speedup (CPU/GPU): %.2fx\n", cpu_time / gpu_time);

    // --- Cleanup ---
    free(h_A);
    free(h_B);
    free(h_C_cpu);
    free(h_C_gpu);
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
```

```
    return 0;
}
```
